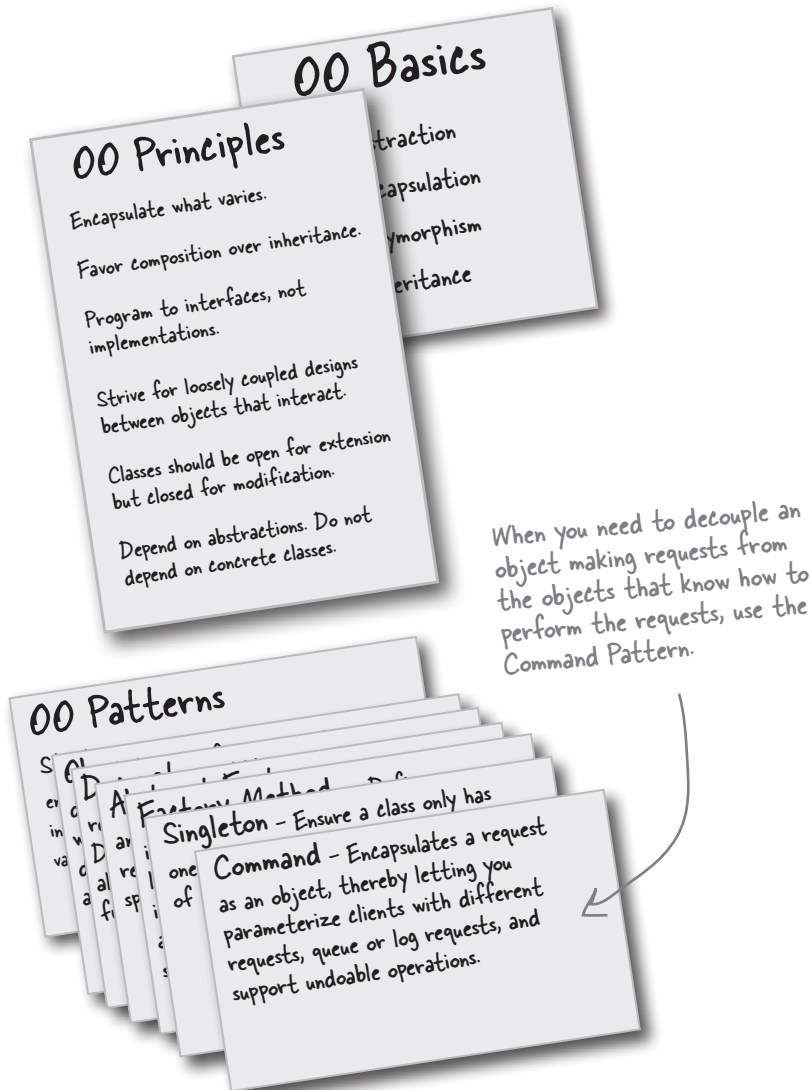




Tools for your Design Toolbox

Your toolbox is starting to get heavy! In this chapter we've added a pattern that allows us to encapsulate methods into Command objects: store them, pass them around, and invoke them when you need them.



BULLET POINTS

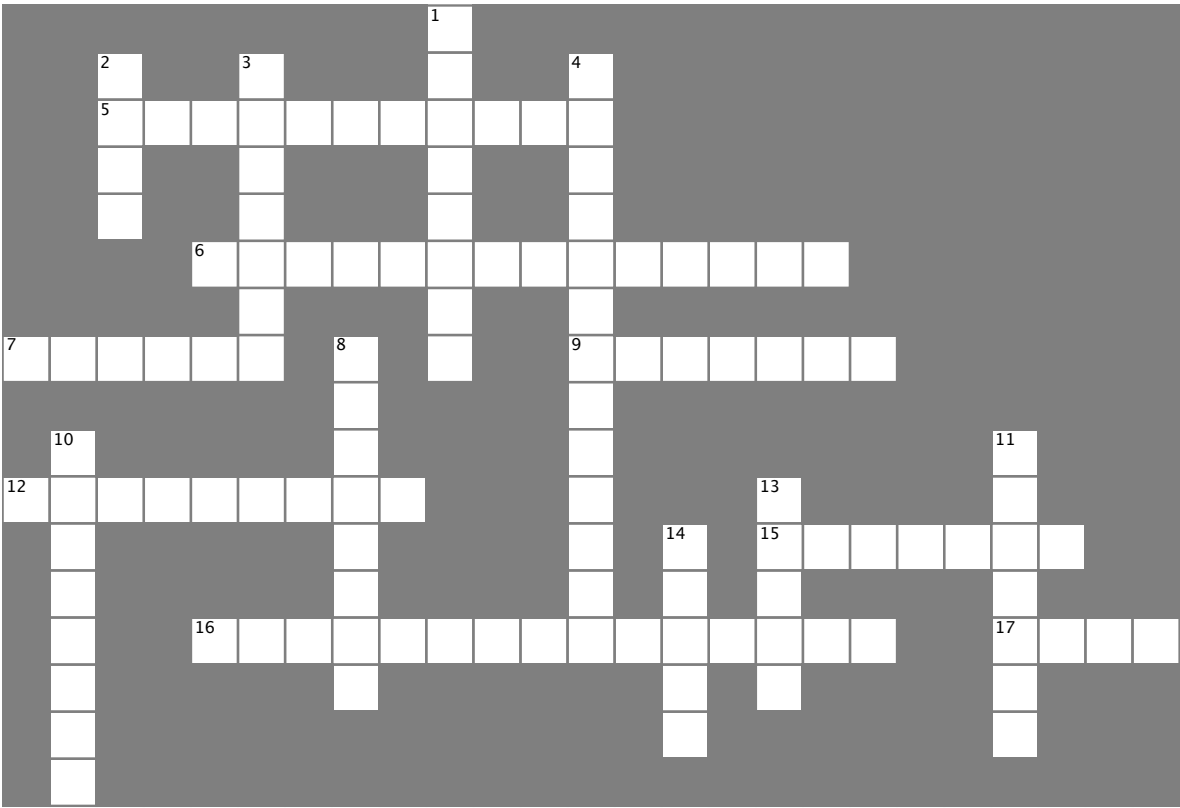
- The Command Pattern decouples an object making a request from the one that knows how to perform it.
- A Command object is at the center of this decoupling and encapsulates a receiver with an action (or set of actions).
- An invoker makes a request of a Command object by calling its `execute()` method, which invokes those actions on the receiver.
- Invokers can be parameterized with Commands, even dynamically at runtime.
- Commands may support undo by implementing an `undo()` method that restores the object to its previous state before the `execute()` method was last called.
- MacroCommands are a simple extension of the Command Pattern that allow multiple commands to be invoked. Likewise, MacroCommands can easily support `undo()`.
- In practice, it's not uncommon for “smart” Command objects to implement the request themselves rather than delegating to a receiver.
- Commands may also be used to implement logging and transactional systems.



Design Patterns Crossword

Time to take a breather and let it all sink in.

It's another crossword; all of the solution words are from this chapter.



ACROSS

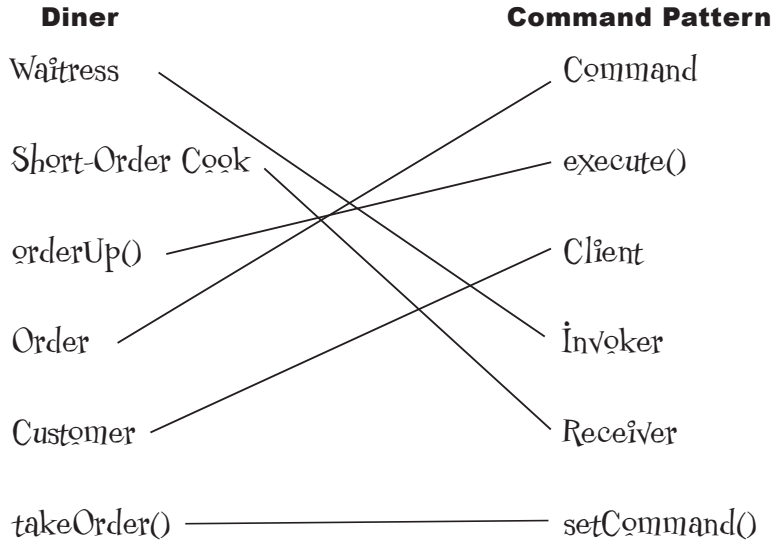
- 5. Our favorite city.
- 6. Company that got us word-of-mouth business.
- 7. Role of customer in the Command Pattern.
- 9. Object that knows the actions and the receiver.
- 12. Invoker and receiver are _____.
- 15. The Waitress was one.
- 16. Dr. Seuss diner food (four words).
- 17. Another thing Command can do.

DOWN

- 1. The Cook and this person were definitely decoupled.
- 2. The Waitress didn't do this.
- 3. A command encapsulates this.
- 4. Act as the receivers in the remote control (two words).
- 8. Object that knows how to get things done.
- 10. Carries out a request.
- 11. All commands provide this.
- 13. Our first command object controlled this.
- 14. A command _____ a set of actions and a receiver.

WHO DOES WHAT? SOLUTION

Match the diner objects and methods with the corresponding names from the Command Pattern.



Sharpen your pencil Solution

Here's the code for the GarageDoorOpenCommand class.

```
public class GarageDoorOpenCommand implements Command {
    GarageDoor garageDoor;

    public GarageDoorOpenCommand(GarageDoor garageDoor) {
        this.garageDoor = garageDoor;
    }

    public void execute() {
        garageDoor.up();
    }
}
```

Here's the output:

```
File Edit Window Help GreenEggs&Ham
%java RemoteControlTest
Light is on
Garage Door is Open
%
```



Exercise Solution

Here is the undo() method for the MacroCommand.

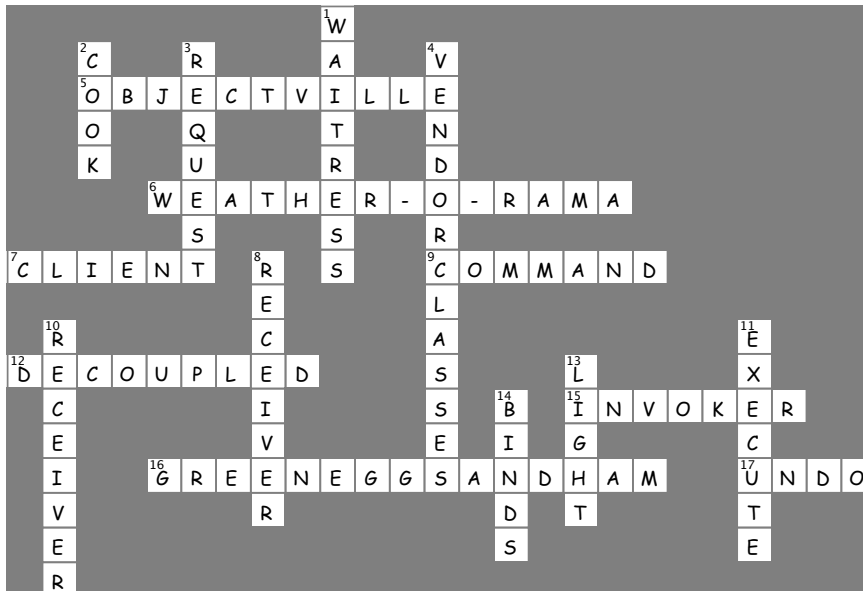
```
public class MacroCommand implements Command {
    Command[] commands;
    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }
    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }
    public void undo() {
        for (int i = commands.length - 1; i >= 0; i--) {
            commands[i].undo();
        }
    }
}
```



Sharpen your pencil Solution

Here's the code to create commands for the off button.

```
LightOffCommand lightOff = new LightOffCommand(light);
StereoOffCommand stereoOff = new StereoOffCommand(stereo);
TVOffCommand tvOff = new TVOffCommand(tv);
HottubOffCommand hottubOff = new HottubOffCommand(hottub);
```



7 the Adapter and Facade Patterns

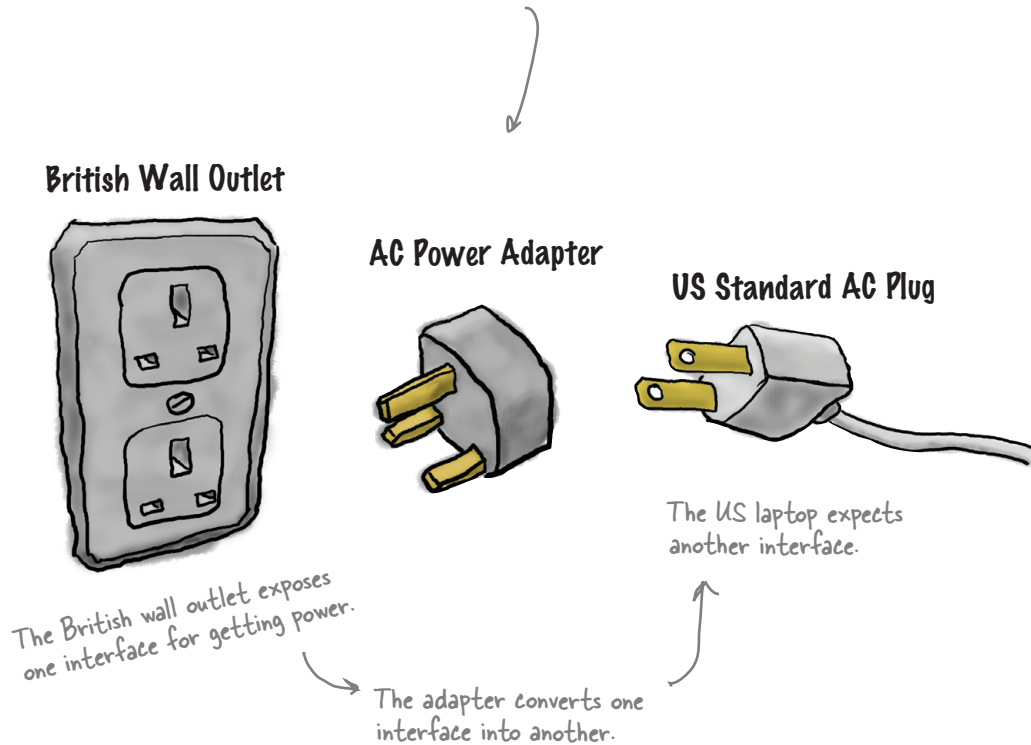
Being Adaptive



In this chapter we're going to attempt such impossible feats as putting a square peg in a round hole. Sound impossible? Not when we have Design Patterns. Remember the Decorator Pattern? We **wrapped objects** to give them new responsibilities. Now we're going to wrap some objects with a different purpose: to make their interfaces look like something they're not. Why would we do that? So we can adapt a design expecting one interface to a class that implements a different interface. That's not all; while we're at it, we're going to look at another pattern that wraps objects to simplify their interface.

Adapters all around us

You'll have no trouble understanding what an OO adapter is because the real world is full of them. How's this for an example: Have you ever needed to use a US-made laptop in Great Britain? Then you've probably needed an AC power adapter...



You know what the adapter does: it sits in between the plug of your laptop and the British AC outlet; its job is to adapt the British outlet so that you can plug your laptop into it and receive power. Or look at it this way: the adapter changes the interface of the outlet into one that your laptop expects.

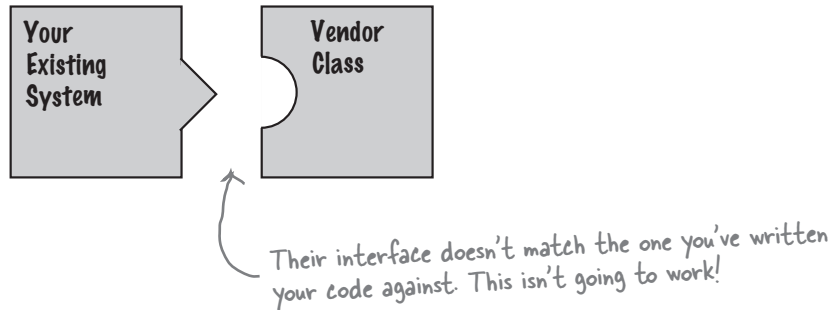
Some AC adapters are simple—they only change the shape of the outlet so that it matches your plug, and they pass the AC current straight through—but other adapters are more complex internally and may need to step the power up or down to match your devices' needs.

Okay, that's the real world; what about object-oriented adapters? Well, our OO adapters play the same role as their real-world counterparts: they take an interface and adapt it to one that a client is expecting.

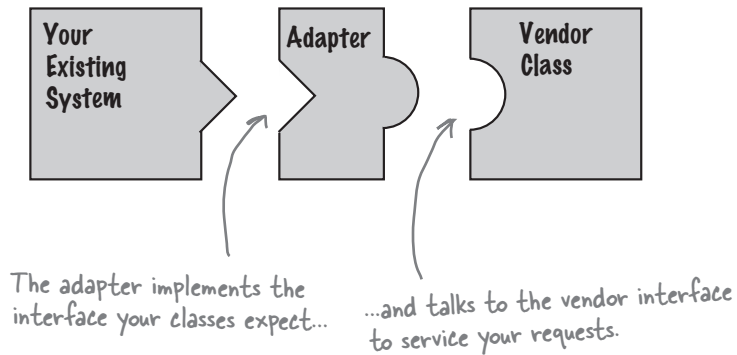
How many other real-world adapters can you think of?

Object-oriented adapters

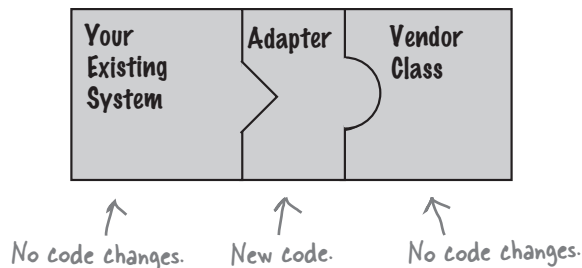
Say you've got an existing software system that you need to work a new vendor class library into, but the new vendor designed their interfaces differently than the last vendor:



Okay, you don't want to solve the problem by changing your existing code (and you can't change the vendor's code). So what do you do? Well, you can write a class that adapts the new vendor interface into the one you're expecting.



The adapter acts as the middleman by receiving requests from the client and converting them into requests that make sense on the vendor classes.



Can you think of a solution that doesn't require **YOU** to write **ANY** additional code to integrate the new vendor classes? How about making the vendor supply the adapter class?

If it walks like a duck and quacks like a duck, then it ~~must~~ might be a ~~duck~~ turkey wrapped with a duck adapter...

It's time to see an adapter in action. Remember our ducks from Chapter 1?
Let's review a slightly simplified version of the Duck interfaces and classes:



```
public interface Duck {  
    public void quack();  
    public void fly();  
}
```

This time around, our
ducks implement a Duck
interface that allows
Ducks to quack and fly.

Here's a subclass of Duck, the MallardDuck:

```
public class MallardDuck implements Duck {  
    public void quack() {  
        System.out.println("Quack");  
    }  
  
    public void fly() {  
        System.out.println("I'm flying");  
    }  
}
```

Simple implementations: MallardDuck
just prints out what it is doing.

Now it's time to meet the newest fowl on the block:

```
public interface Turkey {  
    public void gobble();  
    public void fly();  
}
```

Turkeys don't quack, they gobble.

Turkeys can fly, although they
can only fly short distances.

```

public class WildTurkey implements Turkey {
    public void gobble() {
        System.out.println("Gobble gobble");
    }

    public void fly() {
        System.out.println("I'm flying a short distance");
    }
}

```

Here's a concrete implementation of Turkey; like MallardDuck, it just prints out its actions.

Now, let's say you're short on Duck objects and you'd like to use some Turkey objects in their place. Obviously we can't use the turkeys outright because they have a different interface.

So, let's write an Adapter:



Code Up Close

```

public class TurkeyAdapter implements Duck {
    Turkey turkey;

```

```

    public TurkeyAdapter(Turkey turkey) {
        this.turkey = turkey;
    }

```

```

    public void quack() {
        turkey.gobble();
    }

```

```

    public void fly() {
        for(int i=0; i < 5; i++) {
            turkey.fly();
        }
    }
}

```

First, you need to implement the interface of the type you're adapting to. This is the interface your client expects to see.

Next, we need to get a reference to the object that we are adapting; here we do that through the constructor.

Now we need to implement all the methods in the interface; the quack() translation between classes is easy: just call the gobble() method.

Even though both interfaces have a fly() method, Turkeys fly in short spurts—they can't do long-distance flying like ducks. To map between a Duck's fly() method and a Turkey's, we need to call the Turkey's fly() method five times to make up for it.

Test drive the adapter

Now we just need some code to test drive our adapter:

```
public class DuckTestDrive {
    public static void main(String[] args) {
        Duck duck = new MallardDuck();

        Turkey turkey = new WildTurkey();
        Duck turkeyAdapter = new TurkeyAdapter(turkey);

        System.out.println("The Turkey says...");
        turkey.gobble();
        turkey.fly();

        System.out.println("\nThe Duck says...");
        testDuck(duck);

        System.out.println("\nThe TurkeyAdapter says...");
        testDuck(turkeyAdapter);
    }
}
```

```
static void testDuck(Duck duck) {
    duck.quack();
    duck.fly();
}
}
```

Let's create a Duck...
...and a Turkey.

And then wrap the turkey in a TurkeyAdapter, which makes it look like a Duck.

Then, let's test the Turkey: make it gobble, make it fly.

Now let's test the duck by calling the testDuck() method, which expects a Duck object.

Now the big test: we try to pass off the turkey as a duck...

Here's our testDuck() method; it gets a duck and calls its quack() and fly() methods.

Test run

```
File Edit Window Help Don'tForgetToDuck
%java DuckTestDrive
The Turkey says...
Gobble gobble
I'm flying a short distance

The Duck says...
Quack
I'm flying

The TurkeyAdapter says...
Gobble gobble
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
```

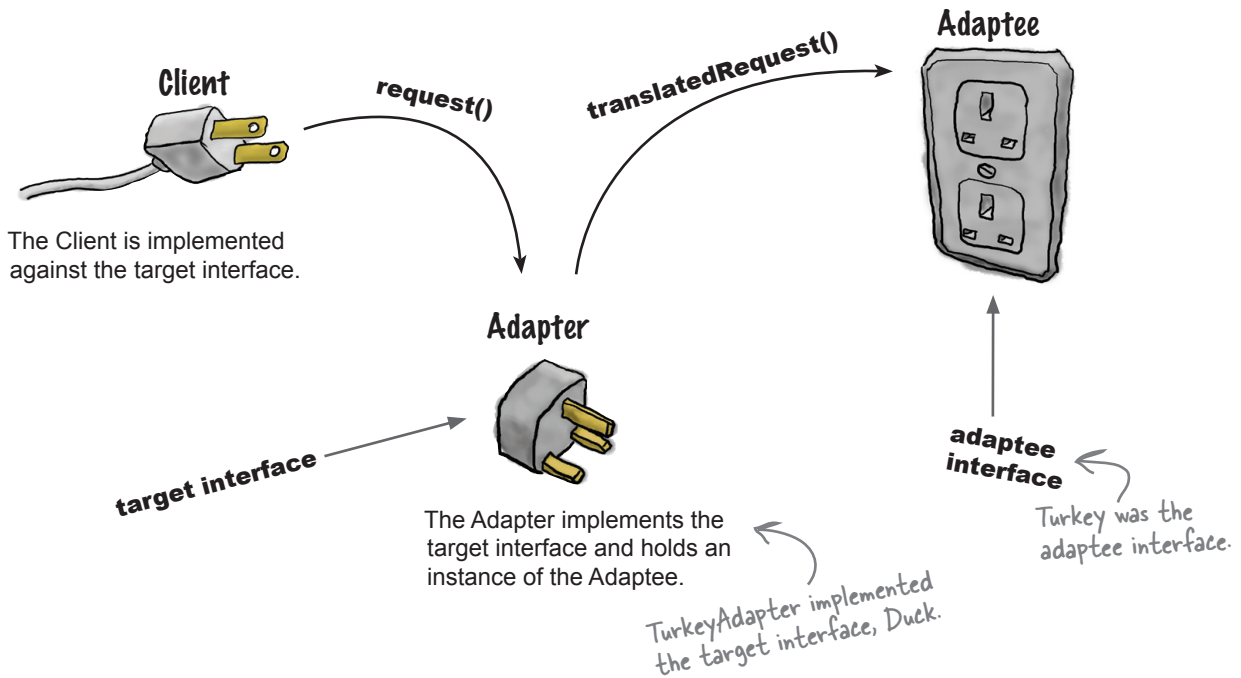
The Turkey gobbles and flies a short distance.

The Duck quacks and flies just like you'd expect.

And the adapter gobbles when quack() is called and flies a few times when fly() is called. The testDuck() method never knows it has a turkey disguised as a duck!

The Adapter Pattern explained

Now that we have an idea of what an Adapter is, let's step back and look at all the pieces again.



Here's how the Client uses the Adapter

- 1 The client makes a request to the adapter by calling a method on it using the target interface.
- 2 The adapter translates the request into one or more calls on the adaptee using the adaptee interface.
- 3 The client receives the results of the call and never knows there is an adapter doing the translation.

Note that the Client and Adaptee are decoupled—neither knows about the other.



Let's say we also need an Adapter that converts a Duck to a Turkey. Let's call it DuckAdapter. Write that class:

How did you handle the fly() method (after all, we know ducks fly longer than turkeys)?
Check the answers at the end of the chapter for our solution. Did you think of a better way?

there are no Dumb Questions

Q: How much “adapting” does an adapter need to do? It seems like if I need to implement a large target interface, I could have a LOT of work on my hands.

A: You certainly could. The job of implementing an adapter really is proportional to the size of the interface you need to support as your target interface. Think about your options, however. You could rework all your client-side calls to the interface, which would result in a lot of investigative work and code changes. Or, you can cleanly provide one class that encapsulates all the changes in that class.

Q: Does an adapter always wrap one and only one class?

A: The Adapter Pattern's role is to convert one interface into another. While most examples of the Adapter Pattern show an adapter wrapping one adaptee, we both know the world is often a bit more messy. So, you may well have situations where an adapter holds two or more adaptees that are needed to implement the target interface.

This relates to another pattern called the Facade Pattern; people often confuse the two. Remind us to revisit this point when we talk about facades later in this chapter.

Q: What if I have old and new parts of my system, and the old parts expect the old vendor interface, but we've already written the new parts to use the new vendor interface? It's going to get confusing using an adapter here and the unwrapped interface there. Wouldn't I be better off just writing my older code and forgetting the adapter?

A: Not necessarily. One thing you can do is create a Two Way Adapter that supports both interfaces. To create a Two Way Adapter, just implement both interfaces involved, so the adapter can act as an old interface or a new interface.

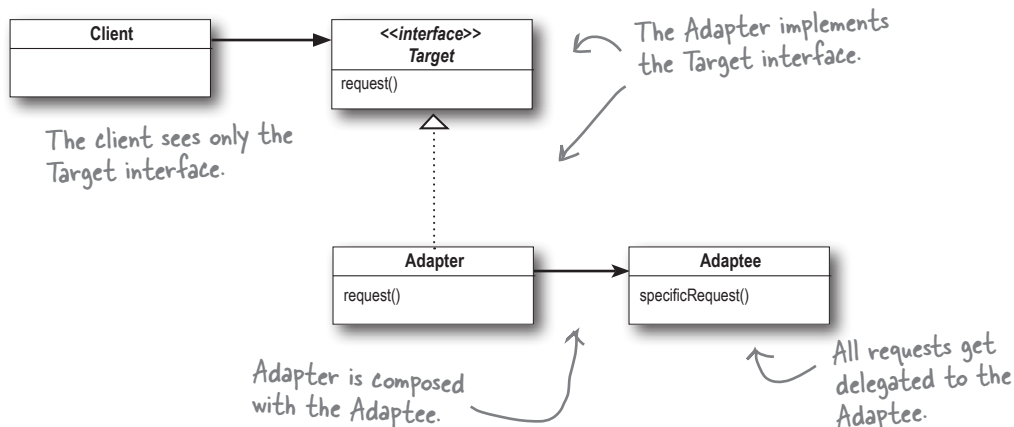
Adapter Pattern defined

Enough ducks, turkeys, and AC power adapters; let's get real and look at the official definition of the Adapter Pattern:

The Adapter Pattern converts the interface of a class into another interface the clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.

Now, we know this pattern allows us to use a client with an incompatible interface by creating an Adapter that does the conversion. This acts to decouple the client from the implemented interface, and if we expect the interface to change over time, the adapter encapsulates that change so that the client doesn't have to be modified each time it needs to operate against a different interface.

We've taken a look at the runtime behavior of the pattern; let's take a look at its class diagram as well:



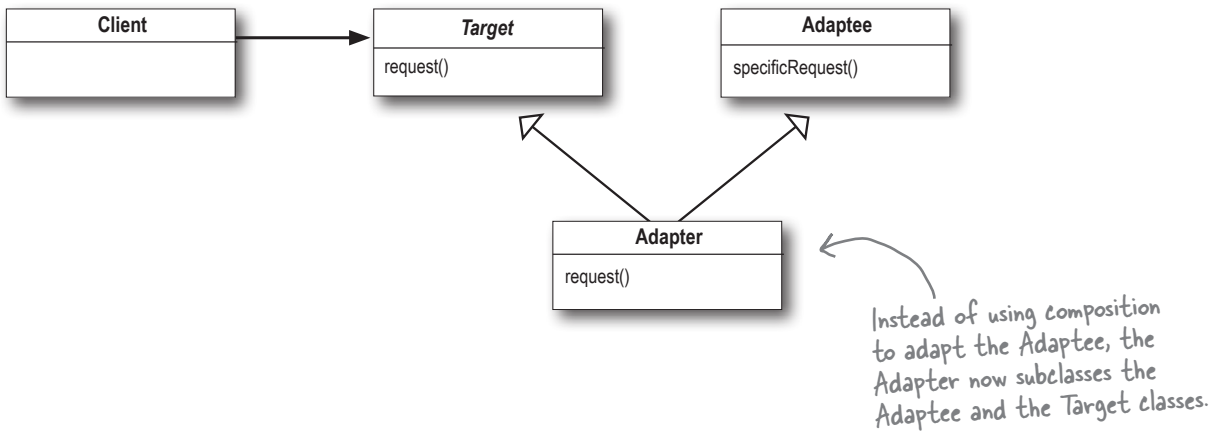
The Adapter Pattern is full of good object-oriented design principles: check out the use of object composition to wrap the adaptee with an altered interface. This approach has the added advantage that we can use an adapter with any subclass of the adaptee.

Also check out how the pattern binds the client to an interface, not an implementation; we could use several adapters, each converting a different backend set of classes. Or, we could add new implementations after the fact, as long as they adhere to the Target interface.

Object and class adapters

Now despite having defined the pattern, we haven't told you the whole story yet. There are actually *two* kinds of adapters: *object* adapters and *class* adapters. This chapter has covered object adapters, and the class diagram on the previous page is a diagram of an object adapter.

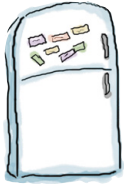
So what's a *class* adapter and why haven't we told you about it? Because you need multiple inheritance to implement it, which isn't possible in Java. But that doesn't mean you might not encounter a need for class adapters down the road when using your favorite multiple inheritance language! Let's look at the class diagram for multiple inheritance.



Look familiar? That's right—the only difference is that with a class adapter we subclass the Target and the Adaptee, while with an object adapter we use composition to pass requests to an Adaptee.



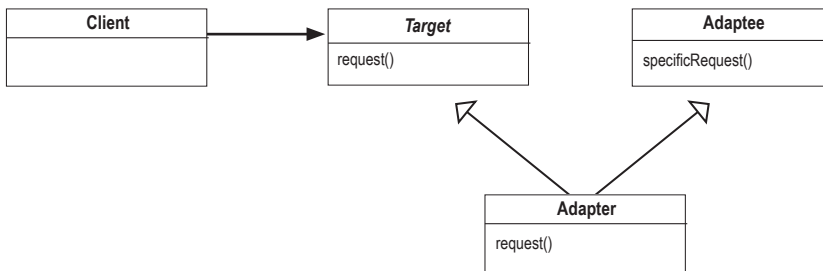
Object adapters and class adapters use two different means of adapting the adaptee (composition versus inheritance). How do these implementation differences affect the flexibility of the adapter?



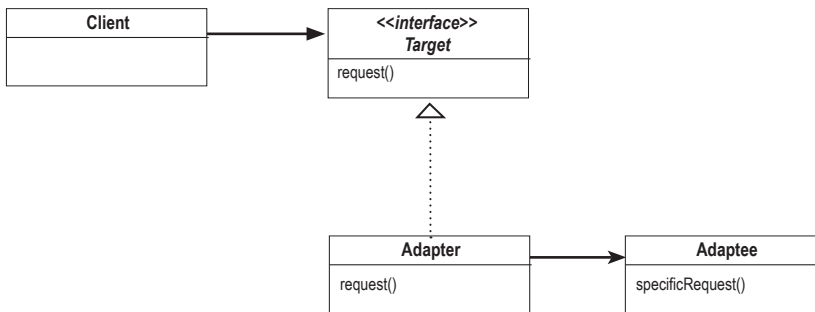
Duck Magnets

Your job is to take the duck and turkey magnets and drag them over the part of the diagram that describes the role played by that bird, in our earlier example. (Try not to flip back through the pages.) Then add your own annotations to describe how it works.

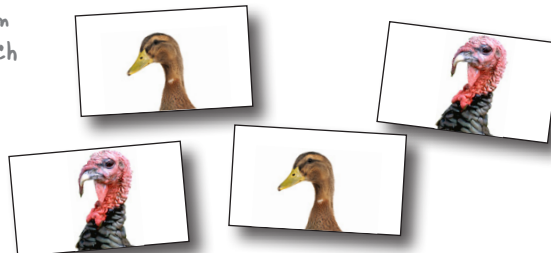
Class Adapter



Object Adapter



Drag these onto the class diagram to show which part of the diagram represents the Duck class and which represents the Turkey class.

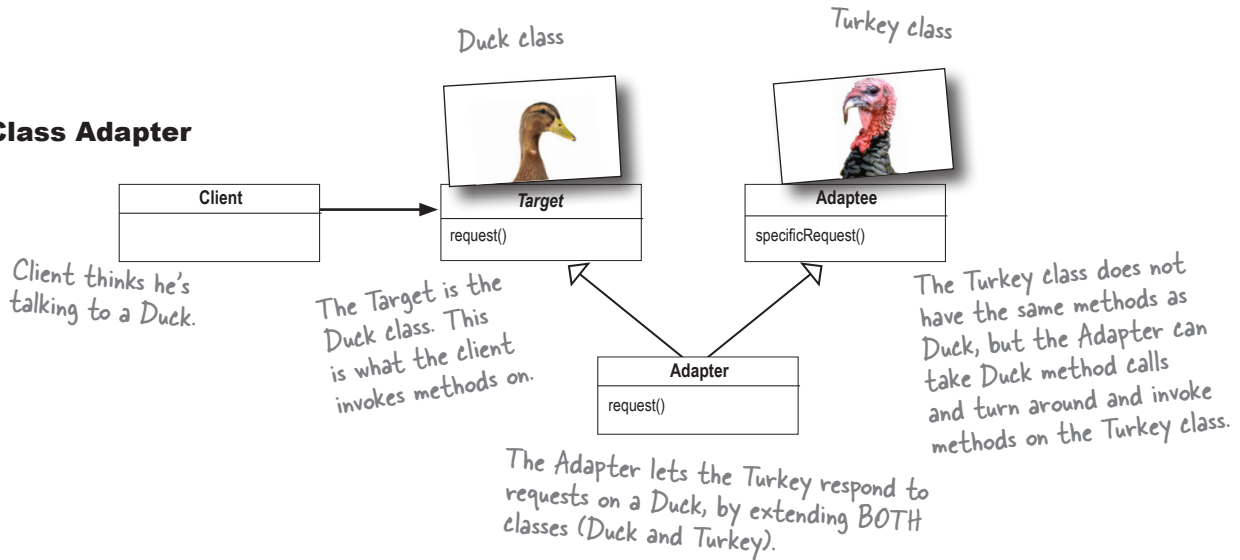




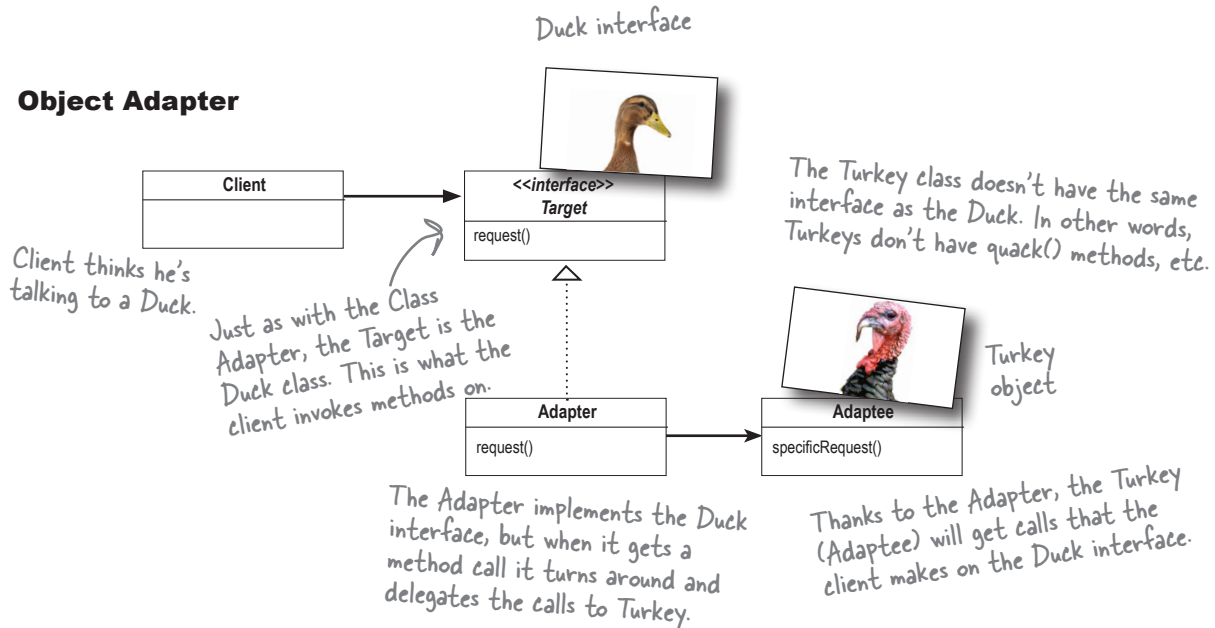
Duck Magnets Answer

Note: the class adapter uses multiple inheritance, so you can't do it in Java...

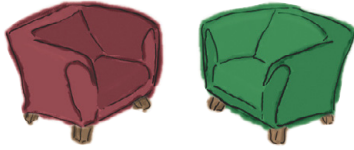
Class Adapter



Object Adapter



Fireside Chats



Tonight's talk: **Object Adapter and Class Adapter meet face to face.**

Object Adapter:

Because I use composition I've got a leg up. I can adapt not only an adaptee class, but any of its subclasses.

In my part of the world, we like to use composition over inheritance; you may be saving a few lines of code, but all I'm doing is writing a little code to delegate to the adaptee. We like to keep things flexible.

You're worried about one little object? You might be able to quickly override a method, but any behavior I add to my adapter code works with my adaptee class *and* all its subclasses.

Hey, come on, cut me some slack, I just need to compose with the subclass to make that work.

You wanna see messy? Look in the mirror!

Class Adapter:

That's true, I do have trouble with that because I am committed to one specific adaptee class, but I have a huge advantage because I don't have to reimplement my entire adaptee. I can also override the behavior of my adaptee if I need to because I'm just subclassing.

Flexible maybe, but efficient? No. There is just one of me, not an adapter and an adaptee.

Yeah, but what if a subclass of Adaptee adds some new behavior—then what?

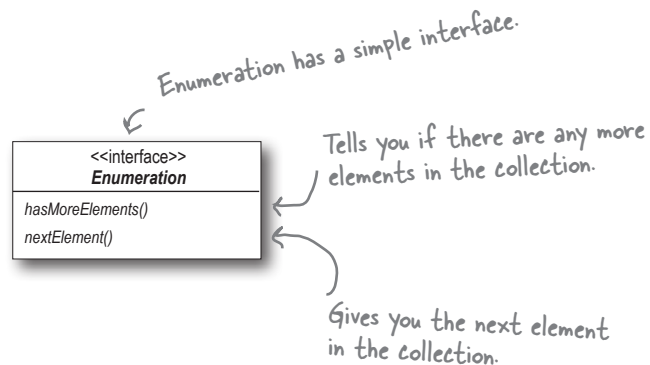
Sounds messy...

Real-world adapters

Let's take a look at the use of a simple Adapter in the real world (something more serious than Ducks at least)...

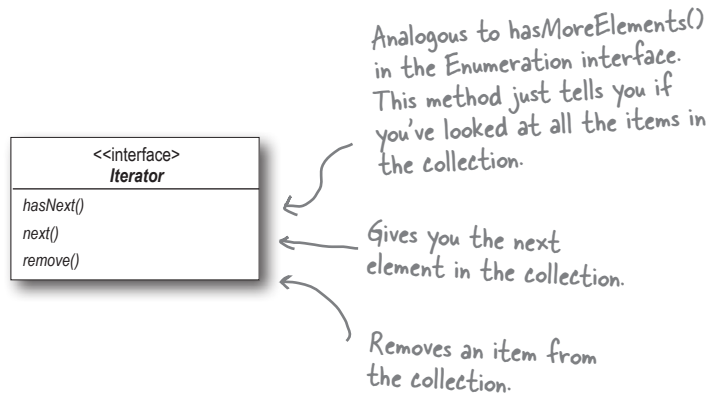
Enumerators

If you've been around Java for a while, you probably remember that the early collection types (Vector, Stack, Hashtable, and a few others) implement a method, `elements()`, which returns an Enumeration. The Enumeration interface allows you to step through the elements of a collection without knowing the specifics of how they are managed in the collection.



Iterators

The more recent Collection classes use an Iterator interface that, like the Enumeration interface, allows you to iterate through a set of items in a collection, and adds the ability to remove items.

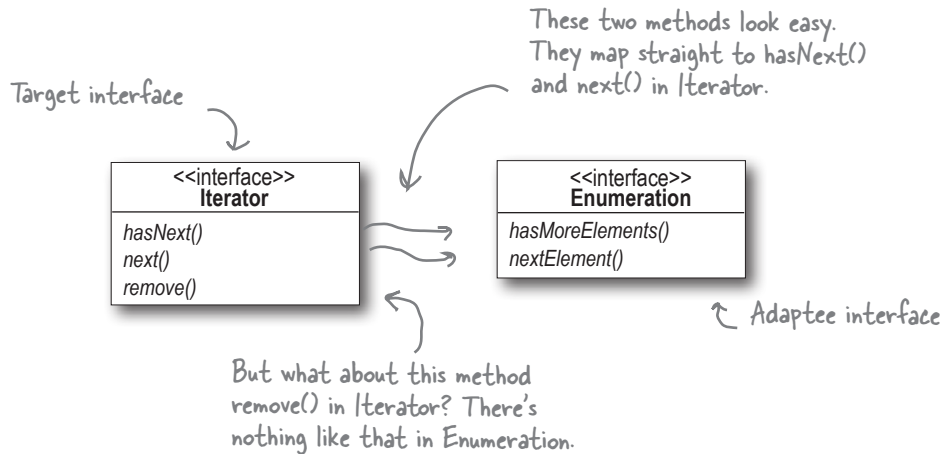


Using Enumerators with code that expects Iterators

We are sometimes faced with legacy code that exposes the Enumeration interface, yet we'd like for our new code to use only Iterators. It looks like we need to build an adapter.

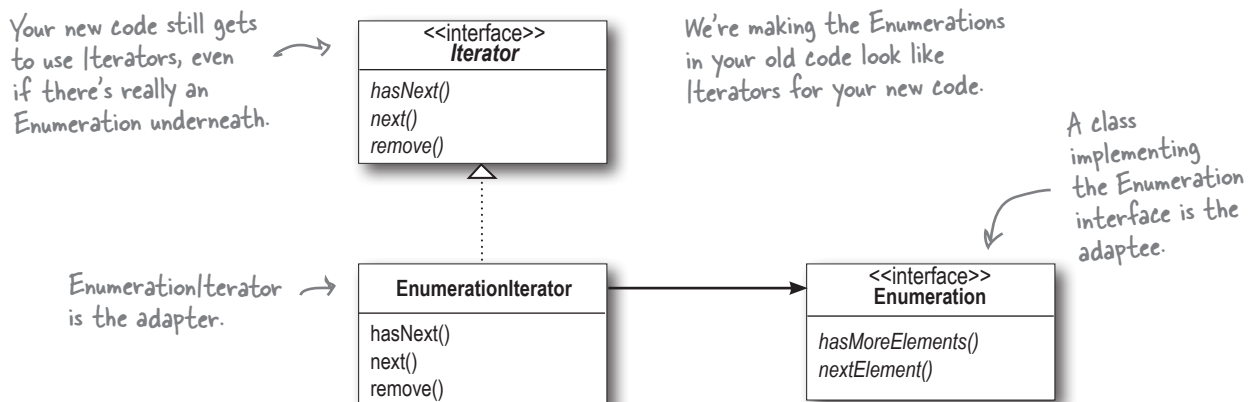
Adapting an Enumeration to an Iterator

First we'll look at the two interfaces to figure out how the methods map from one to the other. In other words, we'll figure out what to call on the adaptee when the client invokes a method on the target.



Designing the Adapter

Here's what the classes should look like: we need an adapter that implements the Target interface and is composed with an adaptee. The hasNext() and next() methods are going to be straightforward to map from target to adaptee: we just pass them right through. But what do you do about remove()? Think about it for a moment (and we'll deal with it on the next page). For now, here's the class diagram:



Dealing with the remove() method

Well, we know Enumeration doesn't support remove(). It's a "read only" interface. There's no way to implement a fully functioning remove() method on the adapter. The best we can do is throw a runtime exception. Luckily, the designers of the Iterator interface foresaw this need and defined the remove() method so that it supports an UnsupportedOperationException.

This is a case where the adapter isn't perfect; clients will have to watch out for potential exceptions, but as long as the client is careful and the adapter is well documented, this is a perfectly reasonable solution.

Writing the EnumerationIterator adapter

Here's simple but effective code for all those legacy classes still producing Enumerations:

```
public class EnumerationIterator implements Iterator<Object> {
    Enumeration<?> enumeration;

    public EnumerationIterator(Enumeration<?> enumeration) {
        this.enumeration = enumeration;
    }

    public boolean hasNext() {
        return enumeration.hasMoreElements();
    }

    public Object next() {
        return enumeration.nextElement();
    }

    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

Since we're adapting Enumeration to Iterator, our Adapter implements the Iterator interface...it has to look like an Iterator.

The Enumeration we're adapting. We're using composition, so we stash it in an instance variable.

The Iterator's hasNext() method is delegated to the Enumeration's hasMoreElements() method...

...and the Iterator's next() method is delegated to the Enumeration's nextElement() method.

Unfortunately, we can't support Iterator's remove() method, so we have to punt (in other words, we give up!). Here we just throw an exception.



Exercise

While Java has gone in the direction of the `Iterator` interface, there is nevertheless still legacy client code that depends on the `Enumeration` interface, so an Adapter that converts an `Iterator` to an `Enumeration` could potentially be useful.

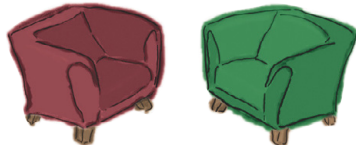
Write an Adapter that adapts an `Iterator` to an `Enumeration`. You can test your code by adapting an `ArrayList`. The `ArrayList` class supports the `Iterator` interface but doesn't support `Enumerations`.



Some AC adapters do more than just change the interface—they add other features like surge protection, indicator lights, and other bells and whistles.

If you were going to implement these kinds of features, what pattern would you use?

Fireside Chats



Tonight's talk: **The Decorator Pattern and the Adapter Pattern discuss their differences.**

Decorator:

I'm important. My job is all about *responsibility*—you know that when a Decorator is involved, there's going to be some new responsibilities or behaviors added to your design.

That may be true, but don't think we don't work hard. When we have to decorate a big interface, whoa, that can take a lot of code.

Cute. Don't think we get all the glory; sometimes I'm just one decorator that is being wrapped by who knows how many other decorators. When a method call gets delegated to you, you have no idea how many other decorators have already dealt with it and you don't know that you'll ever get noticed for your efforts servicing the request.

Adapter:

You decorators want all the glory while us adapters are down in the trenches doing the dirty work: converting interfaces. Our jobs may not be glamorous, but our clients sure do appreciate us making their lives simpler.

Try being an adapter when you've got to bring several classes together to provide the interface your client is expecting. Now that's tough. But we have a saying: "A decoupled client is a happy client."

Hey, if adapters are doing their job, our clients never even know we're there. It can be a thankless job.

Decorator:

Well, us decorators do that as well, only we allow *new behavior* to be added to classes without altering existing code. I still say that adapters are just fancy decorators—I mean, just like us, you wrap an object.

Uh, no. Our job in life is to extend the behaviors or responsibilities of the objects we wrap; we aren't a *simple pass through*.

Maybe we should agree to disagree. We seem to look somewhat similar on paper, but clearly we are *miles* apart in our *intent*.

Adapter:

But the great thing about us adapters is that we allow clients to make use of new libraries and subsets without changing *any* code; they just rely on us to do the conversion for them. Hey, it's a niche, but we're good at it.

No, no, no, not at all. We *always* convert the interface of what we wrap; you *never* do. I'd say a decorator is like an adapter; it's just that you don't change the interface!

Hey, who are you calling a simple pass through? Come on down and we'll see how long *you* last converting a few interfaces!

Oh yeah, I'm with you there.

And now for something different...

There's another pattern in this chapter.

You've seen how the Adapter Pattern converts the interface of a class into one that a client is expecting. You also know we achieve this in Java by wrapping the object that has an incompatible interface with an object that implements the correct one.

We're going to look at a pattern now that alters an interface, but for a different reason: to simplify the interface. It's aptly named the Facade Pattern because this pattern hides all the complexity of one or more classes behind a clean, well-lit facade.



Match each pattern with its intent:

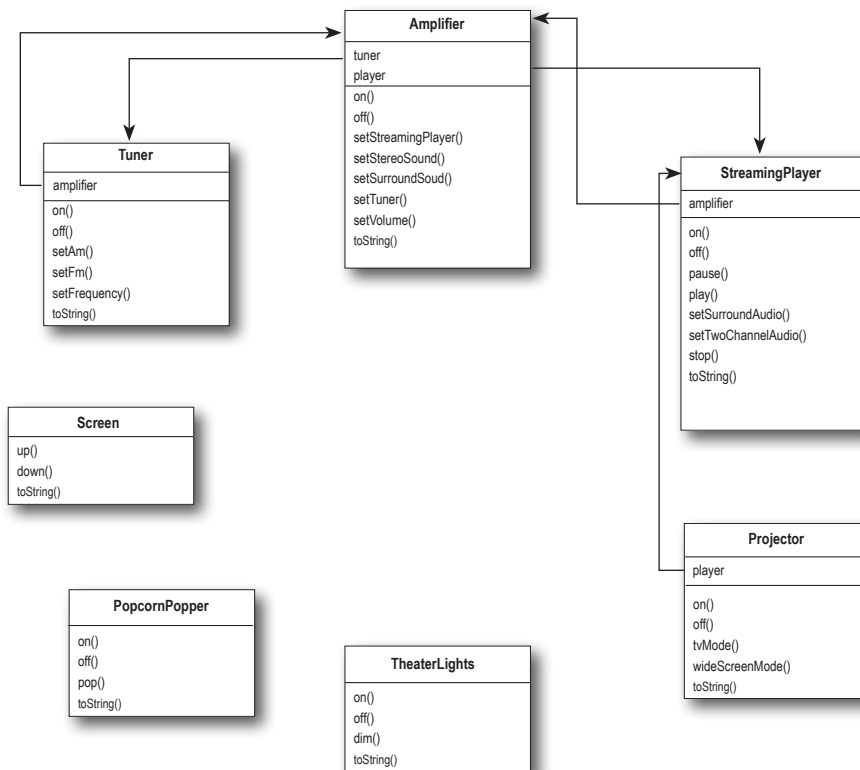
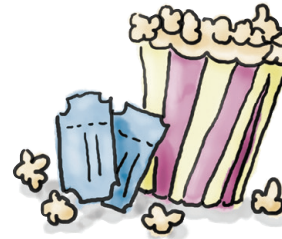
Pattern	Intent
Decorator	Converts one interface to another
Adapter	Doesn't alter the interface, but adds responsibility
Facade	Makes an interface simpler

Home Sweet Home Theater

Before we dive into the details of the Facade Pattern, let's take a look at a growing national obsession: building a nice theater to binge-watch all those movies and TV series.

You've done your research and you've assembled a killer system complete with a streaming player, a projection video system, an automated screen, surround sound, and even a popcorn popper.

Check out all the components you've put together:



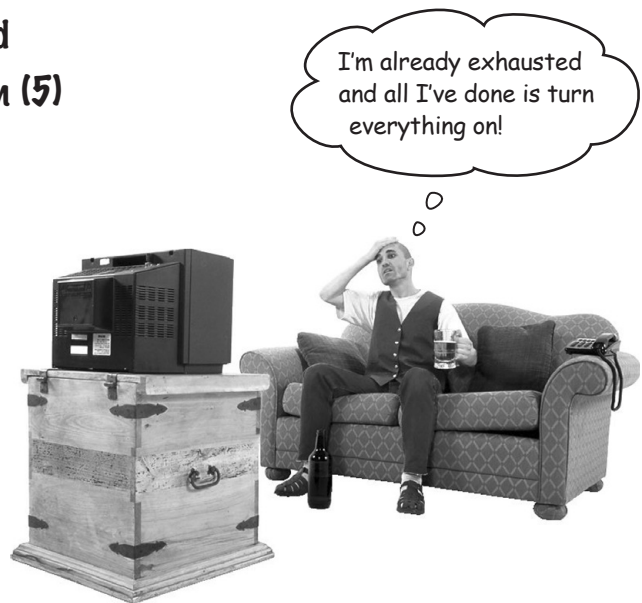
That's a lot of classes, a lot of interactions, and a big set of interfaces to learn and use.

You've spent weeks running wire, mounting the projector, making all the connections, and fine tuning. Now it's time to put it all in motion and enjoy a movie...

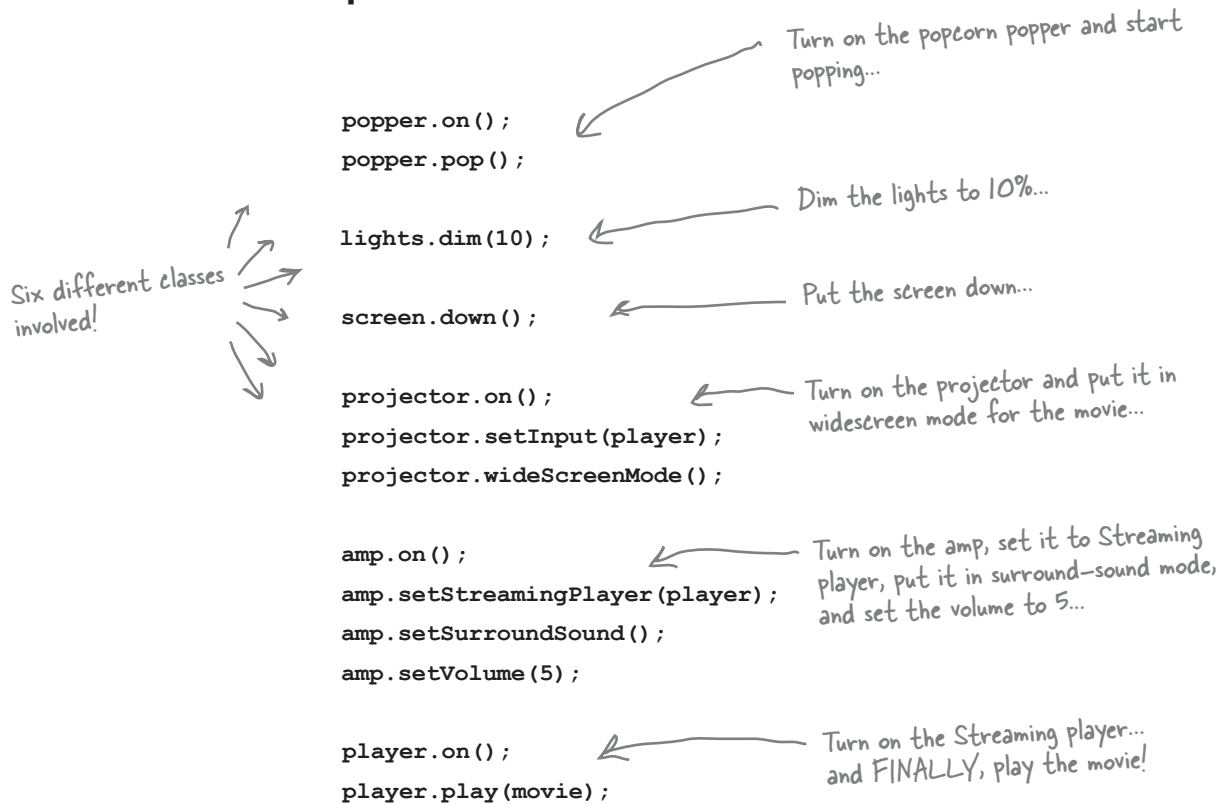
Watching a movie (the hard way)

Pick out a movie, relax, and get ready for movie magic. Oh, there's just one thing—to watch the movie, you need to perform a few tasks:

- ❶ Turn on the popcorn popper**
- ❷ Start the popper popping**
- ❸ Dim the lights**
- ❹ Put the screen down**
- ❺ Turn the projector on**
- ❻ Set the projector input to streaming player**
- ❼ Put the projector on widescreen mode**
- ❽ Turn the sound amplifier on**
- ❾ Set the amplifier to streaming player input**
- ❿ Set the amplifier to surround sound**
- ⓫ Set the amplifier volume to medium (5)**
- ⓬ Turn the streaming player on**
- ⓭ Start playing the movie**



Let's check out those same tasks in terms of the classes and the method calls needed to perform them:



But there's more...

- When the movie is over, how do you turn everything off? Wouldn't you have to do all of this over again, in reverse?
- Wouldn't it be as complex to listen to the radio?
- If you decide to upgrade your system, you're probably going to have to learn a slightly different procedure.

So what to do? The complexity of using your home theater is becoming apparent!

Let's see how the Facade Pattern can get us out of this mess so we can enjoy the movie...

Lights, Camera, Facade!

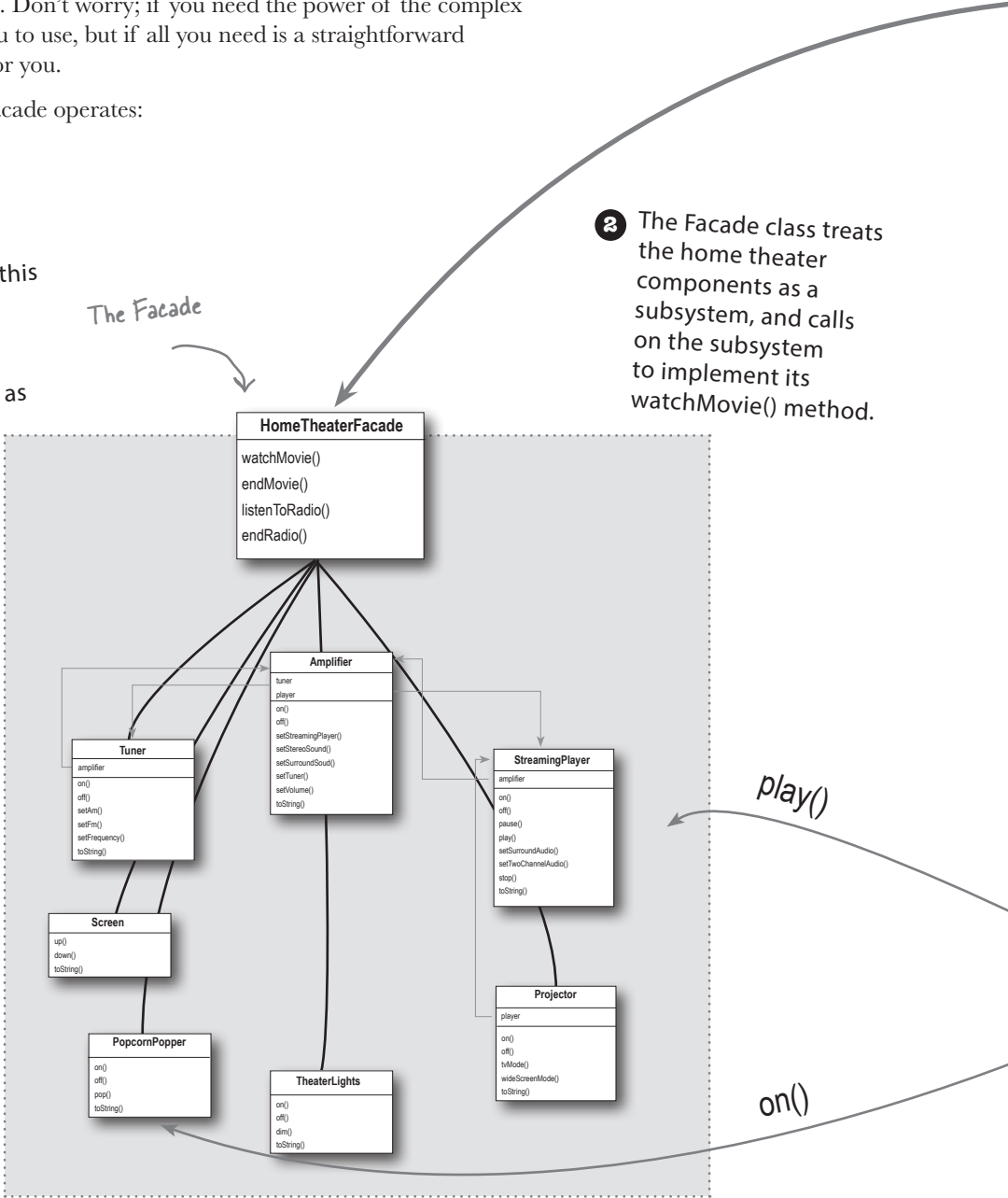
A Facade is just what you need: with the Facade Pattern you can take a complex subsystem and make it easier to use by implementing a Facade class that provides one, more reasonable interface. Don't worry; if you need the power of the complex subsystem, it's still there for you to use, but if all you need is a straightforward interface, the Facade is there for you.

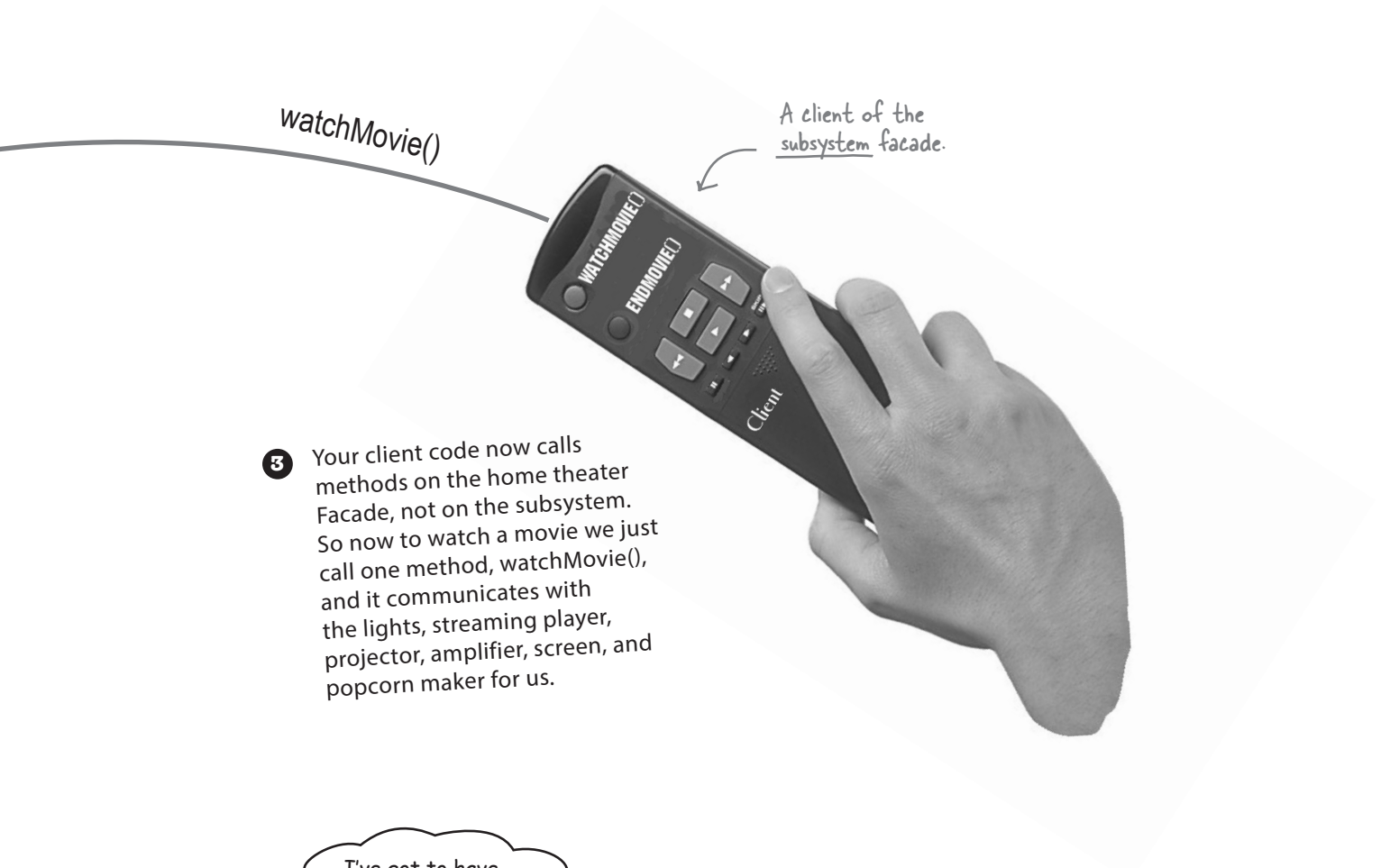
Let's take a look at how the Facade operates:

1 Okay, time to create a Facade for the home theater system. To do this we create a new class HomeTheaterFacade, which exposes a few simple methods such as watchMovie().

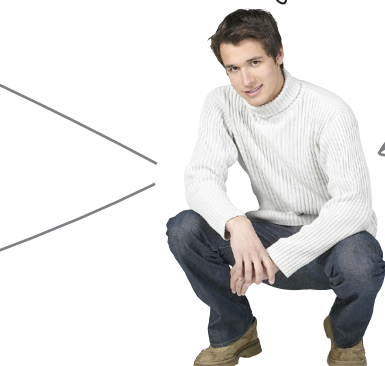
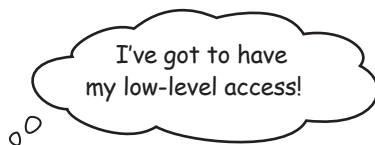
2 The Facade class treats the home theater components as a subsystem, and calls on the subsystem to implement its watchMovie() method.

The subsystem the Facade is simplifying.





- 3** Your client code now calls methods on the home theater Facade, not on the subsystem. So now to watch a movie we just call one method, `watchMovie()`, and it communicates with the lights, streaming player, projector, amplifier, screen, and popcorn maker for us.



Former president of the
Rushmore High School
A/V Science Club.

- 4** The Facade still leaves the subsystem accessible so it can be used directly. If you need the advanced functionality of the subsystem classes, they are available for your use.

there are no Dumb Questions

Q: If the facade encapsulates the subsystem classes, how does a client that needs lower-level functionality gain access to them?

A: Facades don't "encapsulate" the subsystem classes; they merely provide a simplified interface to their functionality. The subsystem classes still remain available for direct use by clients that need to use more specific interfaces. This is a nice property of the Facade Pattern: it provides a simplified interface while still exposing the full functionality of the system to those who may need it.

Q: Does the facade add any functionality or does it just pass through each request to the subsystem?

A: A facade is free to add its own "smarts" in addition to making use of the subsystem. For instance, while our home theater facade doesn't implement any new behavior, it is smart enough to know that the popcorn popper has to be turned on before it can pop (as well as the details of how to turn on and stage a movie showing).

Q: Does each subsystem have only one facade?

A: Not necessarily. The pattern certainly allows for any number of facades to be created for a given subsystem.

Q: What is the benefit of the facade other than the fact that I now have a simpler interface?

A: The Facade Pattern also allows you to decouple your client implementation from any one subsystem. Let's say that you get a big raise and decide to upgrade your home theater to all new components that have different interfaces. Well, if you coded your client to the facade rather than the subsystem, your client code doesn't need to change, just the facade (and hopefully the manufacturer is supplying that!).

Q: So the way to tell the difference between the Adapter Pattern and the Facade Pattern is that the adapter wraps one class and the facade may represent many classes?

A: No! Remember, the Adapter Pattern changes the interface of one or more classes into one interface that a client is expecting. While most textbook examples show the adapter adapting one class, you may need to adapt many classes to provide the interface a client is coded to. Likewise, a Facade may provide a simplified interface to a single class with a very complex interface.

The difference between the two is not in terms of how many classes they "wrap," it is in their intent. The intent of the Adapter Pattern is to alter an interface so that it matches one a client is expecting. The intent of the Facade Pattern is to provide a simplified interface to a subsystem.

A facade not only simplifies an interface, it decouples a client from a subsystem of components.

Facades and adapters may wrap multiple classes, but a facade's intent is to simplify, while an adapter's is to convert the interface to something different.